
OOE - Übungen

Dr. Thomas Nindel

SS 2026

The Meta

- Lösen Sie die folgenden Aufgaben mit Papier und Stift; Bildschirme nur zum Nachschlagen verwenden (Folien, Java-Syntax ist ok, ABER KEINEN COMPILER und keine AI!
 - Es gibt nicht immer eine einzige richtige Antwort
 - Zeit: 120 Minuten
-

Erwärmung 1: Vererbung vs. Komposition

Kategorisieren Sie die folgenden Beziehungen als Vererbung (Ist-ein) oder Komposition (Hat-ein):

- Küche und Toaster
 - Toaster und Küchengerät
 - Koch und Messer
 - VeganeMahlzeit und Mahlzeit
 - Kühlschrank und Kompressor
-

Erwärmung 2: Fernbedienung

Stellen Sie sich vor es gibt ein Interface `FernBedienbar`
(`RemoteControllable`)

- Nennen Sie drei sehr unterschiedliche Dinge, die das Interface implementieren würden, vielleicht je eines aus dem Haushalt, aus einer Fabrik, aus einer Webapplikation
 - Was wären Methoden, die alle Dinge, die Fernbedienbar sind, gemeinsam haben?
-

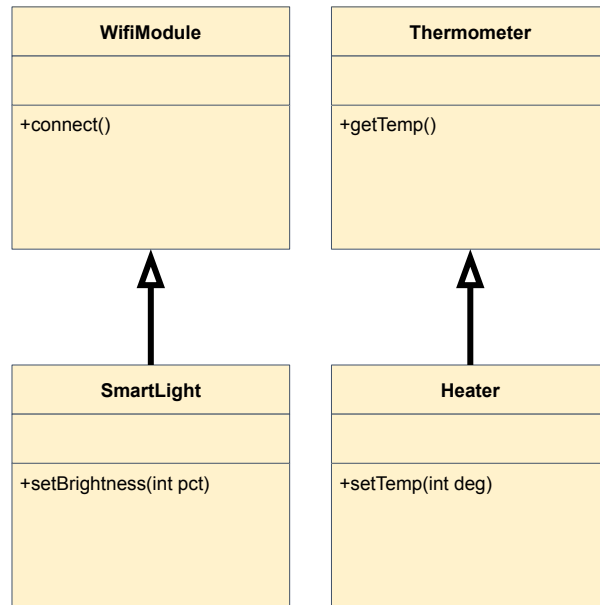
Smart-Home Refactoring

Ein Junior-Entwickler stellt dieses Klassendiagramm vor. Es funktioniert, scheint aber nicht ganz richtig zu sein. Seine Idee: Ein `Heater` muss wie ein `Thermometer` die Temperatur messen können, und zusätzlich auch das Einstellen erlauben. Das `SmartLight` muss wie seine Oberklasse eine Verbindung zum WLAN herstellen, und zusätzlich die Helligkeit steuern können.

1) Kritisieren Sie diesen Entwurf! Warum ist `Heater` als Spezialisierung von gefährlich? Macht es Sinn, wenn z.B. eine Methode `calibrate(Thermometer t)` mit einem `Heater` als Parameter aufgerufen wird (was syntaktisch legal ist)?

2) Ändern Sie das Design und verwenden Sie Delegation statt Vererbung zu Funktionserweiterung und skizzieren Sie, wie ein `Heater` ein `Thermometer` benutzt, ohne selbst eines zu sein!

3) Fügen Sie ein Interface “`Serviceable`” hinzu (Funktion: “`reportStatus()`”) - wo passt das hin?



Late-Binding vs. Early-Binding

1) Output von:

```
item.show();  
item.print(msg);  
item.print(text);
```

?

2) Wann (Laufzeit / Compilezeit)
wurde in jeder Zeile die
Entscheidung getroffen,
welcher Code ausgeführt wird?

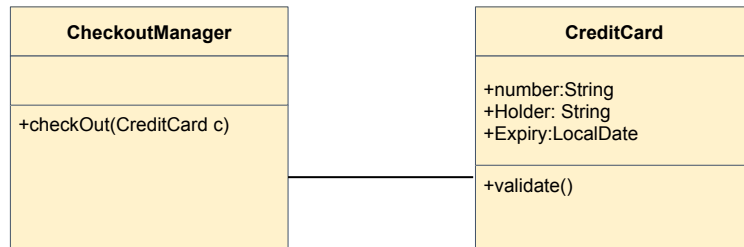
3) Wir ändern die 1. Deklaration
auf Beta item = new Beta(); –
Was passiert jetzt bei
item.print(text)?

```
class Alpha {  
    void print(Object o) { System.out.println("Alpha-Obj"); }  
    void show() { System.out.println("Alpha-Show"); }  
}  
  
class Beta extends Alpha {  
    void print(String s) { System.out.println("Beta-String"); }  
    @Override void show() { System.out.println("Beta-Show"); }  
}  
  
// Execution code:  
Alpha item = new Beta();  
Object msg = "Hello";  
String text = "World";
```

Payment-System

Sie sollen das Payment-System, `CheckoutManager`, für einen Onlineshop entwerfen. Momentan wird nur `CreditCard` unterstützt.

- 1) Wie würden Sie das System um eine neue Zahlungsweise, `PayPal`, erweitern?
- 2) Wie kann `CheckoutManager` eine Zahlung veranlassen ohne zu wissen, welches Zahlungsmittel verwendet wird?
- 3) Zeichnen Sie ein UML-Diagramm mit einer Generalisierung, `PaymentMethod`



Typensicherer Frachter

Ein `Frachter` hat mehrere `FrachtZone`'n. Wir wollen verhindern, dass nicht versehentlich ein `Biohazard` in einer `NahrungsmittelZone` einsortiert wird.

- 1) Entwerfen Sie die Klasse `FrachtZone<T>`
- 2) Entwerfen Sie ein Interface `Verderblich` (`getExpirationDate()`) und ändern Sie `FrachtZone` so, dass es nur Typen akzeptiert, die dieses Interface implementieren
Hint: `class FrachtZone<T extends ...>`

Bei dieser Aufgabe können Sie einen Compiler zur Hilfe nehmen
